these columns to any particular table, this is yet another example of Rails making it easy for a common convention to be implemented easily and consistently.

Options for Creating Tables

You can pass a hash of options as a second parameter to `create_table`. If you specify `force: true`, the migration will drop an existing table of the same name before creating the new one. This is a useful option if you want to create a migration that forces a database into a known state, but there's clearly a potential for data loss.

The `temporary: true` option creates a temporary table—one that goes away when the application disconnects from the database. This is clearly pointless in the context of a migration, but as we'll see later, it does have its uses elsewhere.

The `options: "xxxx"` parameter lets you specify options to your underlying database. They're added to the end of the `CREATE TABLE` statement, right after the closing parenthesis. Although this is rarely necessary with SQLite 3, it may at times be useful with other database servers. For example, some versions of MySQL allow you to specify the initial value of the autoincrementing `id` column. We can pass this in through a migration as follows:

```
create_table :tickets, options: "auto_increment = 10000" do |t|
  t.text :description
  t.timestamps
end
```

Behind the scenes, migrations will generate the following DDL from this table description when configured for MySQL:

```
CREATE TABLE "tickets" (
  "id" int(11) default null auto_increment primary key,
  "description" text,
```